

Slotify: Automated Academic Timetable Generation

Senior Project



Primary Advisor: **Dr. Nazim Ashraf**
Secondary Advisor: **Sharoon Nasim**

Presented by:

261939880

Abial Zafar

261937391

Alisha Saleem

261939549

Sheril Bobby

261940660

Muhammad Zeeshan

Department of Computer Science

Forman Christian College (A Chartered University)

Slotify:Automated Academic Timetable Generation

By

**Abial Zafar
Alisha Saleem
Sheril Bobby
Muhammad Zeeshan**

Project submitted to

Department of Computer Science,
Forman Christian College (A Chartered University),
Lahore, Pakistan.

in partial fulfillment of the requirements for the degree of

**BACHELOR OF SCIENCE
IN
COMPUTER SCIENCE (Honors)**

Primary Project Advisor

Secondary Project Advisor

Senior Project Management
Committee Representative

Abstract

Academic Timetable generation in Forman Christian College's Intermediate Program is currently being done manually by means of arranging lectures, cross checking, swapping timeslots when conflicts so undoubtedly it becomes a very labor intensive task that is inefficient and causes frustration. This project, Slotify, is an intelligent timetable scheduler backed with a Python based constraint satisfaction algorithm that caters to the tedious requirements and constraints for scheduling 3000 lectures for a week. This system allow scheduling for teachers, room allocation and subjects. The frontend is Next.js with TypeScript, allowing better development experiences and streamlining client and server side. This approach aims to alleviate scheduling burden from academic coordinators who are currently doing this process manually and also demonstrate how a constraint satisfying algorithm can be formulated and create a system with various case related functions such as manual overriding, exporting timetables etc.

Acknowledgement

We would like to acknowledge and thank our Primary Advisor, Dr. Nazim Ashraf for providing guidance and feedback throughout this project in all of its iterations. His validation and encouragement were invaluable for the progress of this development.

We are also thankful to our Secondary Advisor, Mr. Sharoon Nasim, for being readily available whenever we needed assistance. His prompt responses and answers to our questions helped us navigate various stages of the project.

Within our team, we would like to acknowledge Abial Zafar, our team lead, for identifying the problem, presenting it to the team, and taking on the responsibility of presenting the foundation of the project. He also worked closely on the backend with Alisha and relayed Forman Christian College's time table methodology for Alisha to then translate to an algorithm.

We extend our appreciation to Alisha Saleem for his extensive work on the backend and his close collaboration with Abial in developing the deep logic and algorithms that power the system. He also worked with the frontend developers to coordinate backend-frontend integration and connection.

Muhammad Zeeshan is to be thanked for creating the initial rough prototype of the system, which served as the foundation for the frontend. He was mainly responsible for developing the frontend and continuously improving it throughout the project's development in all iterations. In addition, he provided guidance on design and stylistic choices across different iterations of the system, perfectly simplifying user experience and making user interface and interaction intuitive. Sheril Bobby is to be thanked for transforming the initial prototype into clear and professional presentations throughout the project. Sheril also worked on organizing and compiling the project documentation by assigning sections to team members. She coordinated closely with Zeeshan, Abial, and Alisha to accurately document the system workflow and development process.

The successful completion of this project is all thanks to the members and the advisors who played according to their respective roles and made the project development possible to begin with.

TABLE OF CONTENTS

ABSTRACT	I
ACKNOWLEDGEMENT	II
CHAPTER 1. INTRODUCTION.....	3
1.1 INTRODUCTION.....	3
1.2 OBJECTIVES	3
1.3 PROBLEM STATEMENT	3
1.4 SCOPE	4
CHAPTER 2. REQUIREMENTS ANALYSIS	5
2.1 LITERATURE REVIEW.....	5
2.2 USER CLASSES AND CHARACTERISTICS	6
2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS.....	6
2.4 ASSUMPTIONS AND DEPENDENCIES	7
2.5 FUNCTIONAL REQUIREMENT	7
2.6 USE CASE DIAGRAM.....	19
2.7 NONFUNCTIONAL REQUIREMENTS.....	26
CHAPTER 3. SYSTEM DESIGN	29
3.1 APPLICATION AND DATA ARCHITECTURE.....	29
3.2 INTERACTIONS AND COLLABORATIONS OF COMPONENTS	31
3.3 SYSTEM ARCHITECTURE.....	31
3.4 ARCHITECTURE EVALUATION	32
3.5 INTERFACE OF COMPONENT WITH EXTERNAL ENTITIES	32
3.6 SCREENSHOTS/PROTOTYPE	33
3.7 OTHER DESIGN DETAILS	37
CHAPTER 4. TEST SPECIFICATION AND RESULTS	39
4.1 TEST CASE SPECIFICATION	39
4.2 DEFECT SUMMARY & RECOMMENDATIONS	46
CHAPTER 5. CONCLUSION AND FUTURE WORK	47
5.1 PROJECT SUMMARY	47
5.2 PROBLEMS FACED	47
5.3 FUTURE WORK.....	47
REFERENCES	49
APPENDIX A GLOSSARY	50
APPENDIX B DEPLOYMENT/INSTALLATION GUIDE	51
APPENDIX C USER MANUAL	52
APPENDIX D STUDENT INFORMATION SHEET	53
APPENDIX E PLAGIARISM FREE CERTIFICATE.....	54
APPENDIX F PLAGIARISM REPORT	55

Revision History

Name	Date	Reason For Changes	Version

Chapter 1. Introduction

1.1 Introduction

Academic timetable generation is a complex problem involving multiple **factors**, such as teacher availability, room capacity, laboratory requirements, and subject-specific scheduling rules. At **Forman Christian College's Intermediate Program**, which we are using as the basis for generalizing the whole Intermediate timetable scheduling, this process is done manually by means of assigning lectures to different section based on the subject-specific scheduling requirement and the checking over other sections to determine an optimal slot where a teacher and room can be assigned. With **over 3,000 lectures per week** across five academic streams and **more than 100 class sections**, it is easy to imagine that the manual approach is struggling and causes frustration for the coordinators.

This Final Year Project proposes **Slotify**, an intelligent, web-based **academic timetable generation system** backed up by a **Python-based Constraint Satisfaction Problem (CSP) engine** automates the scheduling process by being easy to use, ensuring optimal resource utilizations and allocation, generalizing the scheduling process by handing over the variables, factors, and constraints over to the backend. Slotify is built on a modern, full-stack architecture consisting of a Python backend, a PostgreSQL database, and a React.js/Next.js frontend.

The system provides access for real academic coordinators, teachers, and administrators, along with users who aim to create multiple timetable for a BISE affiliated institute with their own factors and variables to consider. Notable features of Slotify are automated clash detection, manual timetable adjustments, and exportable schedule formats. The primary objective of Slotify is to deliver a **intuitive, customizable, quick, optimal and user-friendly scheduling solution** tailored to the specific requirements of FCC, with the easily obtainable capabilities of catering to other BISE institutes scheduling and the potential to be scalable for them.

1.2 Objectives

- Alleviate the significant time taken for manual scheduling tasks.
- Provide automated, clash-free timetable generation based on defined constraints.
- Optimize the utilization of institutional resources, including classrooms and faculty time.
- Offer a user-friendly interface for coordinators and teachers to view and manage schedules.

1.3 Problem Statement

Academic institution have multiple factors and considerations to cater to while scheduling different processes and lecture. At Forman Christian College, the Intermediate Program consists of five academic streams, Medical, Engineering, Computer Science, Humanities, and Commerce, each with around 15 sections, resulting in 150 class groups across Year 1 and Year 2. Each sections timetable must align with:

- Subject-wise lecture allocations
- Teacher availability and workload limits
- Classroom capacity and availability
- Shift timings (morning and afternoon)
- Double-period lab requirements

- Special substitutions such as Music/CS replacing TQ for non-Muslim students
- Merge and partition utilization for specific subjects

Currently, timetable creation is managed manually by academic coordinators using a pure intuition approach and making decisions based on what seems to fit in various timeslots which is inefficient. This leads to frequent:

- Teacher or room schedule conflicts resulting in swapping after timeslots have already been declared
- Unbalanced teaching workloads and constant back and forth on determining where lectures should fit
- Poor classroom utilization and gambling on what ends up working
- Delays in releasing the final schedule

1.4 Scope

Slotify is web based scheduling application designed to streamline and organize timetable scheduling using it's constraint satisfying algorithm (CSP) to handle variable and carry out manipulations for optimal schedule generation.

The product's scope covers:

- **Data Management:** Full management and storage of all factors and resources like Teachers, Rooms, Subjects, Departments, Groups, Sections, Sessions (Years), and Shifts.
- **Core Functionality:** A Python-based CSP solver aimed to declare optimal clash free sections taking constraints and other variables entered by user through Excel or submission forms within the app.
- **Constraint Handling:** Enforcement of complex institutional rules through **hard constraints** (e.g., room capacity limits, teacher workload constraints, and prevention of double-booking) and optimization of **soft constraints** (e.g., balanced teaching workloads, merged classes, double-period laboratories, and preferred time slots).
- **User Interaction:** Straight forward and easily comprehensible UI that follows scheduler conventions and provides an intuitive environment.
- **Exports:** Provides a Progress Tracker and a view of data entered by the user to analyze and determine correctness and reports analytics along with provision of exporting (Pdf and Excel)

Chapter 2. Requirements Analysis

2.1 Literature Review

Timetable generation for an educational institutes is a tedious task that requires combinational methods to be able to generate meaningful schedules. We have to account for multiple variables and certain constraints specific to the educational institute to create an optimal solver for this problem. Previous studies generalize this problem with Genetic Algorithms and constraint solvers in academic scheduling. While many institutes rely on external solvers or well-known frameworks, this project adopts a custom Python-based CSP approach to generalize and optimized scheduling based on FCC-specific constraints. The Python engine allows clear cut modeling of rules and easier debugging and adaptation to institutional changes to cater to different institutes.

Genetic Algorithms for Automated Timetable Generation

Genetic Algorithms have been widely utilized for their efficiency in solving complex scheduling problems. Thakare et al.[1] present a system for automated timetable generation using GA, leading to the result of efficient and performative scheduling algorithm. This study provides an eccentric approach combining and referencing genetic mutations with the problem at hand. Multiple utilities and functions are used to consistently optimize the scheduler such as fitness functions, these allow the algorithm to satisfy constraints put on it along with identifying and preventing new conflicts from emerging. The system also emphasizes user-friendly features like JavaScript-based validation for data entry, facilitating practical deployment.

Similarly, another study[2] proposes a GA-based timetable generation system aimed at minimizing scheduling conflicts and enabling the schedule to produce results in a brief amount of time. *It highlights the limitations of classical linear or integer programming techniques for complex university schedules and advocates for the adaptive nature of GA in handling diverse constraints.* The research elaborates on genetic algorithms as they relate to optimizing a scheduling system and analyzing the overall benefits of using GA for this problem. The system reportedly achieves solution qualities ranging from 60% to 80% of the optimal.

OptaPlanner was considered during initial research but was not adopted due to the presence of strict hard constraints and additional custom constraints defined in the Faculty Course Coordination (FCC) requirements rendering it inconsiderable, **OptaPlanner was unable to produce a fully optimized and justified solution for the complete problem.**

Collectively, these studies highlight the effectiveness of custom genetic algorithms in optimizing scheduling techniques.

Based on this information, this project adopts a Python-based backend utilizing **a custom Constraint Satisfaction Problem (CSP) solver** tailored specifically to the academic and administrative constraints of Forman Christian College. This approach offers the most scalability, reliability and customization as it pertains to the various constraints really well and offers control over generalizing the problem to work beyond the hard constraint on FCC when needed.

2.2 User Classes and Characteristics

The system is designed for different classes of users within the educational institution, each with distinct needs and privileges.

User Class	Characteristics	Need/Benefit	Priority
Academic Coordinators (Users)	Possess the highest security and privilege level; responsible for all setup, generation, and manual overrides; requires detailed understanding of all scheduling constraints.	Efficient, constraint-compliant timetable generation with full control and manual override capability.	Favored/High
Administration Department	Requires access to analytics on resource usage (rooms, faculty) and final timetables for administrative planning; may have read-only access to most features.	Better utilization of rooms and faculty; reduction in scheduling workload and operational inefficiencies.	Medium

2.3 Design and Implementation Constraints

The development of Slotify is bound by specific technical and institutional constraints.

- **Technology Stack Mandate:** Python-based FAST API and React.js/Next.js/Tailwind CSS.
- **Database Requirement:** PostgreSQL as the primary relational database.
- **Institutional Constraints (Hard and Soft):** The solution must be in accordance with the academic rules of Forman Christian College, including:
 - **Hard Constraints:** Preventing teacher/room double-booking, respecting maximum teacher workload, honoring room capacity, and ensuring double-period lab sessions are scheduled once per week.
 - **Soft Constraints:** Balancing teacher workloads and minimizing gaps in teacher schedules.
- **User Interface Standards:** The frontend must be **responsive** and support **Progressive Web App (PWA)** features for mobile access.

2.4 Assumptions and Dependencies

Following are the assumptions and dependencies upon which the success of this project hinges on.

- **Availability of the Python-Based Constraint Solver:**

This project assumes that the algorithmically formulated CSP solver backend is optimized enough to generate timetables in relatively decent timeframe.

- **Data Accuracy:**

For the system, to generate sensible output, the input data (such as teachers, departments etc) give to the system must be sensible and accurate as well.

- **Institutional Cooperation:**

This project depends on constant cooperation from the coordinators of Forman Christian College and their availability to provide feedback, general inferences and considerations for the betterment of the system.

- **Cloud Service Reliability:**

The cloud service used for deployment fully determines the overall availability and reliability of our services.

- **Technology Stability:**

The project assumes that the technologies used, such as the Python backend and the Next based frontend , are able to effectively integrate with one another and remain supported.

2.5 Functional Requirement

2.5.1 Name of Use-Case 1: Login

Identifier	UC-1
Purpose	Allow registered users to authenticate and access the system.
Priority	High
Pre-conditions	User has a registered account. System is operational.
Post-conditions	User is logged in and redirected to the Home Page.

o Typical Course of Action

S#	Actor Action	System Response
1	User enters username and password.	System validates input format.
2	User clicks Login button.	System verifies credentials.
3	-	System grants access and redirects user to Home Page.

o Alternate Course of Action

S#	Actor Action	System Response
1	User enters incorrect credentials.	System displays Invalid username or password.
2	User leaves required fields empty.	System asks user to complete required fields.

2.5.2 Name of Use-Case 2: Sign Up

Identifier	UC-2
Purpose	Register a new user account.
Priority	Medium
Pre-conditions	User is not registered.
Post-conditions	New account is created and user can log in.

o Typical Course of Action

S#	Actor Action	System Response
1	User opens Sign Up page.	System displays registration form.
2	User enters required account details.	System validates required fields.
3	User submits form.	System creates account and redirects to login/home.

o Alternate Course of Action

S#	Actor Action	System Response
1	User enters duplicate username/email.	System rejects registration and displays error.
2	User leaves required fields empty.	System highlights missing fields.

2.5.3 Name of Use-Case 3: Logout

Identifier	UC-3
Purpose	Allow the logged-in user to securely exit the application.
Priority	Medium
Pre-conditions	User is logged in.
Post-conditions	User session is ended and login page is shown.

o Typical Course of Action

S#	Actor Action	System Response
1	User clicks Logout.	System clears session/token.
2	-	System redirects user to Login page.

o Alternate Course of Action

S#	Actor Action	System Response
1	Session is already expired.	System redirects user to Login page.

2.5.4 Name of Use-Case 4: View Dashboard

Identifier	UC-4
Purpose	Display saved timetables and main system options.
Priority	High
Pre-conditions	User is logged in.
Post-conditions	Dashboard is displayed with available actions and saved records.

o Typical Course of Action

S#	Actor Action	System Response
1	User opens dashboard/home.	System loads saved timetables and navigation cards.
2	User selects a module.	System opens selected module.

o Alternate Course of Action

S#	Actor Action	System Response
1	No saved timetable exists.	System displays empty state message.
2	Data loading fails.	System shows error/retry message.

2.5.5 Name of Use-Case 5: Manage Departments

Identifier	UC-5
Purpose	Add, view, edit, search, and delete departments.
Priority	High
Pre-conditions	User is logged in.
Post-conditions	Department records are updated in the system.

o Typical Course of Action

S#	Actor Action	System Response
1	User opens Department section.	System displays department list.
2	User adds or edits department details.	System validates and saves record.
3	User searches or deletes a record.	System filters or removes selected record.

o Alternate Course of Action

S#	Actor Action	System Response
1	User submits empty fields.	System prevents saving and displays validation error.
2	User deletes a department in use.	System prevents deletion or warns about linked records.

2.5.6 Name of Use-Case 6: Manage Subjects

Identifier	UC-6
Purpose	Add, view, edit, search, and delete subjects with department mapping.
Priority	High
Pre-conditions	Departments exist in the system.
Post-conditions	Subject records are stored and available for teachers/sections.

o Typical Course of Action

S#	Actor Action	System Response
1	User opens Subject section.	System displays subject list.
2	User enters subject details and department.	System validates and saves subject.
3	User edits/searches/deletes subject.	System updates displayed data accordingly.

o Alternate Course of Action

S#	Actor Action	System Response
1	User selects no department.	System displays required-field error.
2	Subject is linked to timetable data.	System blocks unsafe deletion or shows warning.

2.5.7 Name of Use-Case 7: Manage Teachers

Identifier	UC-7
Purpose	Add, view, edit, search, and delete teachers with subject assignments and workload.
Priority	High
Pre-conditions	Departments and subjects exist.
Post-conditions	Teacher profile, workload, HOD status, and subjects are saved.

o Typical Course of Action

S#	Actor Action	System Response
1	User opens Teacher section.	System displays teacher records.
2	User enters teacher name, department, workload, HOD status, and subjects.	System validates fields and saves teacher.
3	User edits or deletes teacher.	System updates teacher records.

o Alternate Course of Action

S#	Actor Action	System Response
1	User assigns no valid subject.	System prompts to select valid subject.
2	Teacher is linked to generated timetable.	System warns before deletion or prevents unsafe deletion.

2.5.8 Name of Use-Case 8: Manage Rooms

Identifier	UC-8
Purpose	Add, view, edit, search, and delete rooms with block, capacity, and fixed-room rules.
Priority	High
Pre-conditions	User is logged in. Subjects may exist for fixed rooms.
Post-conditions	Room records are available for timetable generation.

o Typical Course of Action

S#	Actor Action	System Response
1	User opens Room section.	System displays room list.
2	User enters room details, capacity, block, and room type/fixed subject.	System validates and saves room.
3	User edits/searches/deletes room.	System updates records.

o Alternate Course of Action

S#	Actor Action	System Response
1	User leaves room name/block empty.	System displays validation error.
2	Room is already assigned in saved timetable.	System warns or blocks deletion as required.

2.5.9 Name of Use-Case 9: Manage Timeslots

Identifier	UC-9
Purpose	Add, view, edit, bulk-add, search, and delete shift-wise timeslots.
Priority	High

Pre-conditions	User is logged in.
Post-conditions	Timeslots are created and used for generation.

o Typical Course of Action

S#	Actor Action	System Response
1	User opens Timeslot section.	System displays timeslots by shift/day.
2	User manually adds or bulk-generates slots.	System validates day, shift, start/end time and slot index.
3	User edits/searches/deletes timeslot.	System updates list.

o Alternate Course of Action

S#	Actor Action	System Response
1	End time is before start time.	System rejects invalid timeslot.
2	Bulk-add range is invalid.	System displays validation error.

2.5.10 Name of Use-Case 10: Manage Sections

Identifier	UC-10
Purpose	Add, view, edit, search, and delete sections with subjects and lecture counts.
Priority	High
Pre-conditions	Subjects, teachers, rooms, and timeslots may exist.
Post-conditions	Sections are saved with shift, group, year, count, advisor, and subject lectures.

o Typical Course of Action

S#	Actor Action	System Response
1	User opens Section section.	System displays section list.
2	User enters section details and adds subjects with theory/lab counts.	System validates and saves section.
3	User edits/searches/deletes section.	System updates data.

o Alternate Course of Action

S#	Actor Action	System Response
1	User adds section without required fields.	System prevents saving.
2	Subject lecture count is missing/invalid.	System displays validation error.

2.5.11 Name of Use-Case 11: Manage Subsections

Identifier	UC-11
Purpose	Create and manage subsections linked to parent sections with subject lecture counts.
Priority	High
Pre-conditions	Parent section exists.
Post-conditions	Subsection is saved and can be scheduled with parent/independent rules.

o Typical Course of Action

S#	Actor Action	System Response
1	User opens Subsection form.	System displays parent section options.
2	User enters subsection details and subjects with theory/lab counts.	System validates and saves subsection.
3	User edits/searches/deletes subsection.	System updates subsection records.

o Alternate Course of Action

S#	Actor Action	System Response
1	Parent section is not selected.	System asks user to select parent section.
2	Invalid subject count is entered.	System rejects record and shows error.

2.5.12 Name of Use-Case 12: Download Excel Templates

Identifier	UC-12
Purpose	Provide Excel templates matching manual data-entry variables.
Priority	Medium
Pre-conditions	User is logged in.
Post-conditions	Template file is downloaded for selected data module.

o Typical Course of Action

S#	Actor Action	System Response
1	User selects a data module template.	System prepares template with exact required columns.
2	User clicks Download Template.	System downloads Excel file.

o Alternate Course of Action

S#	Actor Action	System Response
1	Template generation fails.	System displays error message.
2	User selects no module.	System asks user to select template type.

2.5.13 Name of Use-Case 13: Import Data from Excel

Identifier	UC-13
Purpose	Import departments, subjects, teachers, rooms, timeslots, sections, or subsections using Excel templates.
Priority	High
Pre-conditions	User has downloaded and filled correct template.
Post-conditions	Valid records are imported into the database.

o Typical Course of Action

S#	Actor Action	System Response
1	User chooses import option.	System opens upload control.
2	User uploads Excel file.	System reads and validates template columns.
3	User confirms import.	System saves valid rows and reports result.

o Alternate Course of Action

S#	Actor Action	System Response
----	--------------	-----------------

1	Uploaded file has wrong columns.	System rejects file and shows template mismatch.
2	Some rows contain invalid data.	System reports skipped/failed rows.

2.5.14 Name of Use-Case 14: Search and Paginate Data

Identifier	UC-14
Purpose	Search records and view limited rows with see more/less controls.
Priority	Medium
Pre-conditions	Data exists in selected module.
Post-conditions	Filtered or expanded data list is displayed.

o Typical Course of Action

S#	Actor Action	System Response
1	User types search keyword.	System filters displayed records.
2	User clicks See More.	System displays additional rows.
3	User clicks See Less.	System collapses list to default rows.

o Alternate Course of Action

S#	Actor Action	System Response
1	No matching record found.	System displays no-results message.

2.5.15 Name of Use-Case 15: Generate Timetable

Identifier	UC-15
Purpose	Generate a conflict-free timetable using available academic data and constraints.
Priority	High
Pre-conditions	Required data exists: teachers, subjects, rooms, timeslots, sections, and lecture counts.
Post-conditions	A generated timetable is created and displayed.

o Typical Course of Action

S#	Actor Action	System Response
1	User clicks Generate Timetable.	System validates required data.
2	-	System starts generation job and initializes progress.
3	-	System applies teacher, room, shift, fixed-room, and workload constraints.
4	-	System returns generated timetable to frontend.

o Alternate Course of Action

S#	Actor Action	System Response
1	Required data is missing.	System displays missing-data error.
2	Some lectures cannot be scheduled.	System reports unscheduled items with reason.

2.5.16 Name of Use-Case 16: Track Generation Progress

Identifier	UC-16
Purpose	Show live progress while timetable generation is running.
Priority	High
Pre-conditions	Generation job has started.
Post-conditions	Progress bar updates until generation completes or fails.

o Typical Course of Action

S#	Actor Action	System Response
1	User starts timetable generation.	System creates generation job.
2	Frontend polls progress endpoint.	System returns current percentage/status.
3	Generation completes.	System marks job complete and progress reaches 100%.

o Alternate Course of Action

S#	Actor Action	System Response
1	Job fails during generation.	System marks job failed and displays error.
2	Progress request fails.	Frontend retries or shows connection error.

2.5.17 Name of Use-Case 17: View Generated Timetable

Identifier	UC-17
Purpose	Display timetable in readable section-wise grid format.
Priority	High
Pre-conditions	Timetable has been generated or saved timetable is opened.
Post-conditions	User can view generated schedule.

o Typical Course of Action

S#	Actor Action	System Response
1	User opens generated timetable.	System displays timetable with days vertically and times horizontally.
2	User scrolls through sections.	System shows each section in separate table/grid.

o Alternate Course of Action

S#	Actor Action	System Response
1	No timetable data exists.	System displays empty timetable message.

2.5.18 Name of Use-Case 18: Filter Timetable

Identifier	UC-18
Purpose	Filter generated timetable by section, department, or block/room.
Priority	High
Pre-conditions	Generated timetable exists.
Post-conditions	Filtered timetable view is displayed.

o Typical Course of Action

S#	Actor Action	System Response
1	User selects filter type.	System loads available sections/departments/blocks.
2	User selects filter value.	System displays matching timetable records.

o Alternate Course of Action

S#	Actor Action	System Response
1	Selected filter has no records.	System displays no matching timetable.
2	Filter data is not loaded.	System shows loading/error message.

2.5.19 Name of Use-Case 19: Save Generated Timetable

Identifier	UC-19
Purpose	Save the generated timetable with a user-provided name.
Priority	High
Pre-conditions	Timetable has been generated.
Post-conditions	Timetable is saved and appears on dashboard.

o Typical Course of Action

S#	Actor Action	System Response
1	User clicks Save after generation.	System opens save prompt/modal.
2	User enters timetable name/year.	System validates name.
3	User confirms save.	System stores timetable and shows success message.

o Alternate Course of Action

S#	Actor Action	System Response
1	User leaves name empty.	System asks user to enter a name.
2	Save fails.	System displays failure message and allows retry.

2.5.20 Name of Use-Case 20: View Saved Timetable

Identifier	UC-20
Purpose	Open and view a previously saved timetable from dashboard.
Priority	High
Pre-conditions	At least one timetable is saved.
Post-conditions	Saved timetable is loaded in view mode.

o Typical Course of Action

S#	Actor Action	System Response
1	User opens dashboard saved timetables.	System displays saved timetable cards.
2	User clicks View.	System loads selected timetable.
3	-	System displays timetable grid and filters.

o Alternate Course of Action

S#	Actor Action	System Response
----	--------------	-----------------

1	Selected saved timetable is unavailable.	System displays not-found/error message.
---	--	--

2.5.21 Name of Use-Case 21: Delete Saved Timetable

Identifier	UC-21
Purpose	Remove an unwanted saved timetable from the dashboard.
Priority	Medium
Pre-conditions	Saved timetable exists.
Post-conditions	Selected saved timetable is deleted.

o Typical Course of Action

S#	Actor Action	System Response
1	User clicks Delete on saved timetable.	System asks for confirmation.
2	User confirms deletion.	System deletes timetable and refreshes dashboard.

o Alternate Course of Action

S#	Actor Action	System Response
1	User cancels confirmation.	System keeps timetable unchanged.
2	Delete fails.	System displays error message.

2.5.22 Name of Use-Case 22: Export Timetable as PDF

Identifier	UC-22
Purpose	Download generated or saved timetable as a PDF file.
Priority	High
Pre-conditions	Timetable exists.
Post-conditions	PDF file is generated and downloaded.

o Typical Course of Action

S#	Actor Action	System Response
1	User selects PDF export.	System prepares printable timetable.
2	User confirms download.	System generates and downloads PDF file.

o Alternate Course of Action

S#	Actor Action	System Response
1	No timetable exists.	System disables export or shows error.
2	PDF generation fails.	System displays failure message.

2.5.23 Name of Use-Case 23: Export Timetable as Excel

Identifier	UC-23
Purpose	Download generated or saved timetable as Excel in grid format.
Priority	High
Pre-conditions	Timetable exists.
Post-conditions	Excel file is generated and downloaded in grid format.

o Typical Course of Action

S#	Actor Action	System Response
1	User selects Excel export.	System converts timetable to spreadsheet grid.
2	User confirms download.	System downloads Excel file.

o Alternate Course of Action

S#	Actor Action	System Response
1	No timetable exists.	System shows export unavailable message.
2	Excel generation fails.	System displays failure message.

2.5.24 Name of Use-Case 24: Apply Teacher Constraints

Identifier	UC-24
Purpose	Ensure teachers are assigned only to valid subjects and not double-booked.
Priority	High
Pre-conditions	Teacher subject mappings and workloads exist.
Post-conditions	Teacher assignments are valid in timetable.

o Typical Course of Action

S#	Actor Action	System Response
1	Scheduler selects lecture to place.	System finds teachers mapped to the subject.
2	-	System checks teacher availability, workload, daily load, and clashes.
3	-	System assigns best available teacher.

o Alternate Course of Action

S#	Actor Action	System Response
1	No valid teacher is available.	System marks lecture unscheduled or tries next available slot/teacher.
2	Teacher workload is full.	System shifts section to another eligible teacher where possible.

2.5.25 Name of Use-Case 25: Apply Room and Lab Constraints

Identifier	UC-25
Purpose	Assign rooms according to availability, capacity, fixed rooms, and lab/theory rules.
Priority	High
Pre-conditions	Room data and room-subject rules exist.
Post-conditions	Lectures use correct room/lab without clashes.

o Typical Course of Action

S#	Actor Action	System Response
1	Scheduler selects lecture room.	System checks fixed rooms/labs for subject.

2	-	System checks capacity, room type, block, and availability.
3	-	System assigns suitable room.

o Alternate Course of Action

S#	Actor Action	System Response
1	Fixed room is unavailable.	System tries other allowed room or marks issue.
2	Theory lecture is being considered for lab.	System rejects lab room unless allowed.

2.5.26 Name of Use-Case 26: Apply Section, Shift, and Subsection Constraints

Identifier	UC-26
Purpose	Schedule sections and subsections within correct shift and required lecture structure.
Priority	High
Pre-conditions	Sections, subsections, shifts, and timeslots exist.
Post-conditions	Lectures remain within correct shift and parent/subsection rules.

o Typical Course of Action

S#	Actor Action	System Response
1	Scheduler reads section shift and lecture requirements.	System selects only matching shift timeslots.
2	-	System schedules parent section and related subsection lectures appropriately.
3	-	System avoids gaps and subject conflicts where possible.

o Alternate Course of Action

S#	Actor Action	System Response
1	No valid shift slot is available.	System tries another valid slot or reports unscheduled lecture.
2	Subsection cannot align with parent section.	System reports conflict for review.

2.5.27 Name of Use-Case 27: Manage Class-Based Teacher Assignment

Identifier	UC-27
Purpose	Assign class based/CB responsibility to an eligible teacher already teaching the section.
Priority	Medium
Pre-conditions	Generated or section-teacher assignment data exists.
Post-conditions	Each section has an eligible CB teacher and one teacher has CB of only one section.

o Typical Course of Action

S#	Actor Action	System Response
----	--------------	-----------------

1	System reviews teachers teaching a section.	System lists eligible teachers.
2	User or scheduler assigns CB teacher.	System checks teacher is teaching that section and has no other CB.
3	-	System saves CB assignment.

o Alternate Course of Action

S#	Actor Action	System Response
1	Selected teacher does not teach the section.	System rejects CB assignment.
2	Teacher already has another CB section.	System asks for another teacher.

2.5.28 Name of Use-Case 28: Validate Data Before Saving

Identifier	UC-28
Purpose	Prevent empty or invalid records across all data management modules.
Priority	High
Pre-conditions	User is adding or editing data.
Post-conditions	Only complete and valid records are saved.

o Typical Course of Action

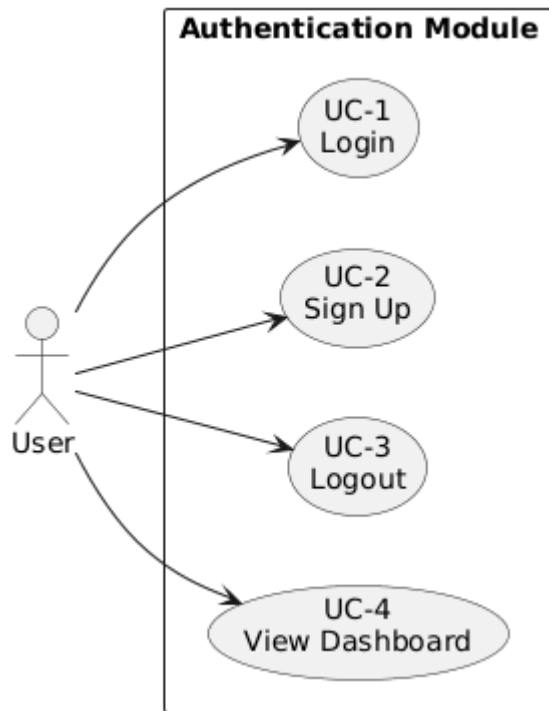
S#	Actor Action	System Response
1	User fills a form.	System checks required fields and valid selections.
2	User submits form.	System saves record if validation passes.
3	-	System clears form state after save/cancel/open.

o Alternate Course of Action

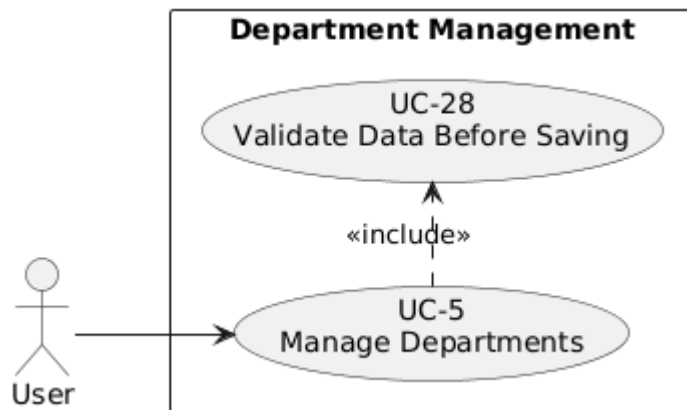
S#	Actor Action	System Response
1	User submits empty fields.	System blocks save and highlights missing fields.
2	Previous form state leaks into new entry.	System resets form state to blank/default values.

2.6 Use Case Diagram

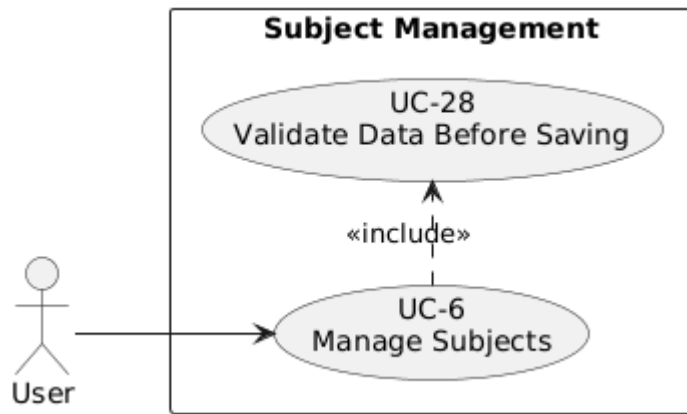
Use case 1,2,3,4:



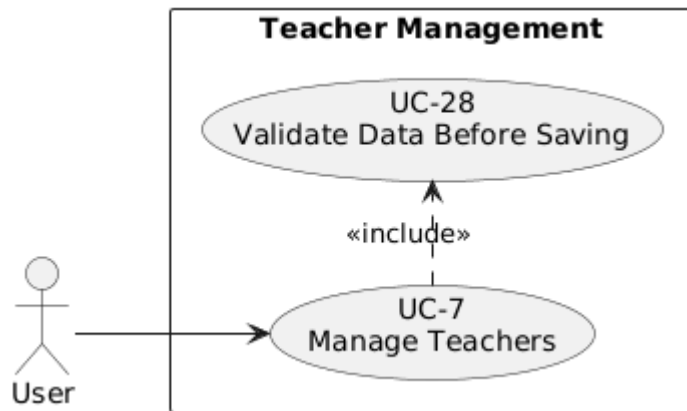
Use case 5:



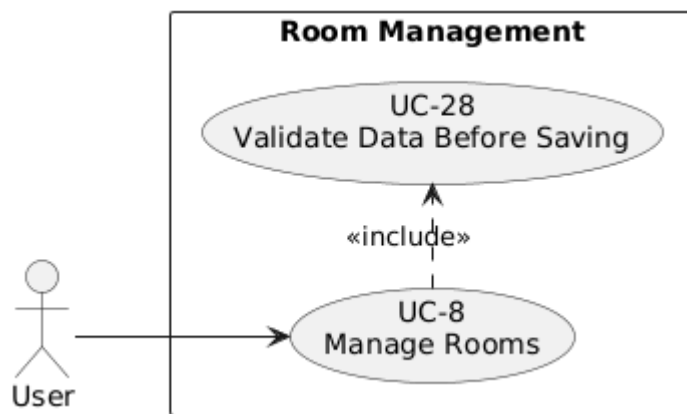
Use case 6:



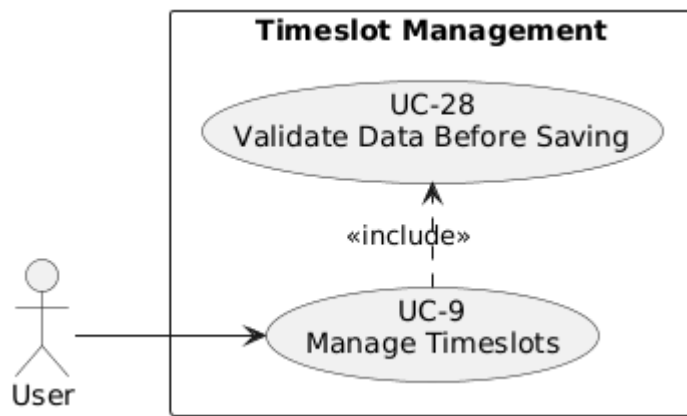
Use case 7:



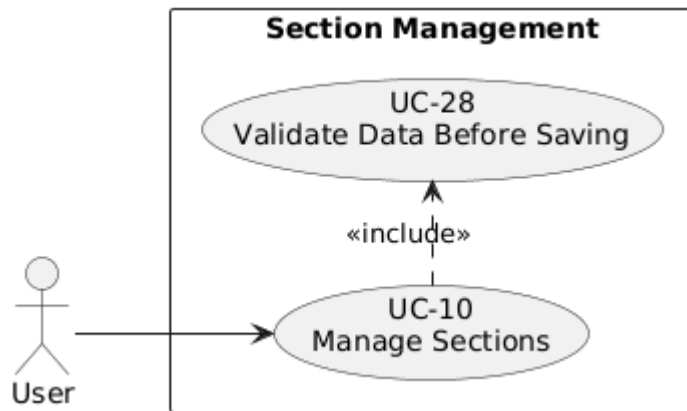
Use case 8:



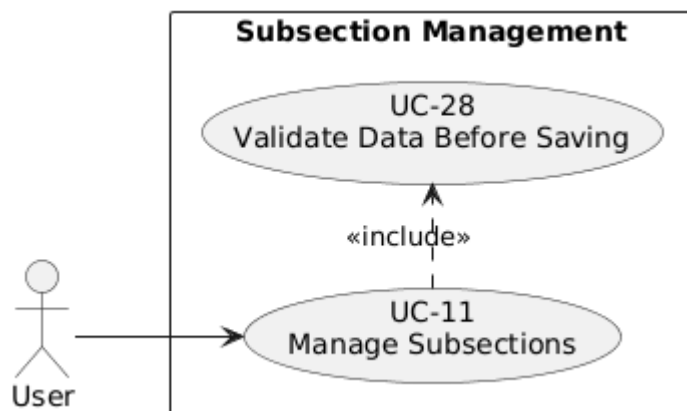
Use case 9:



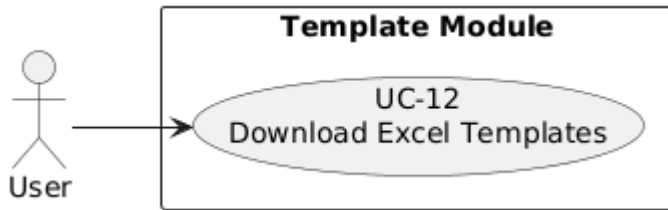
Use case 10:



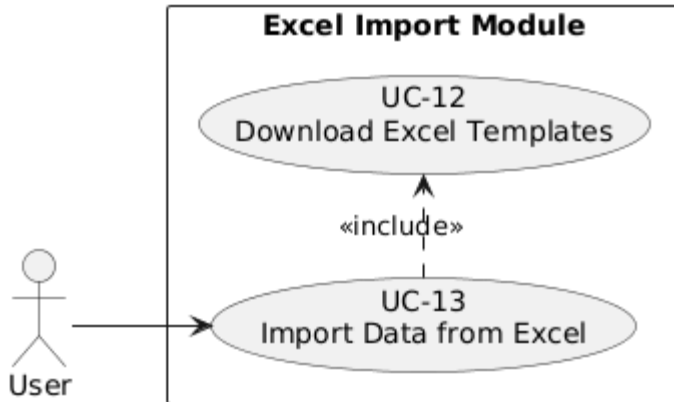
Use case 11:



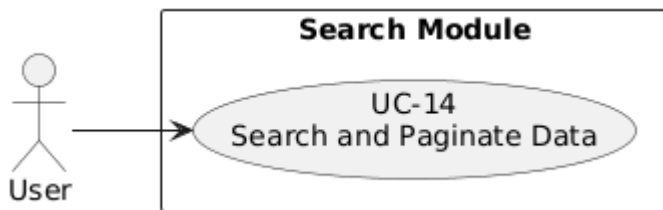
Use case 12:



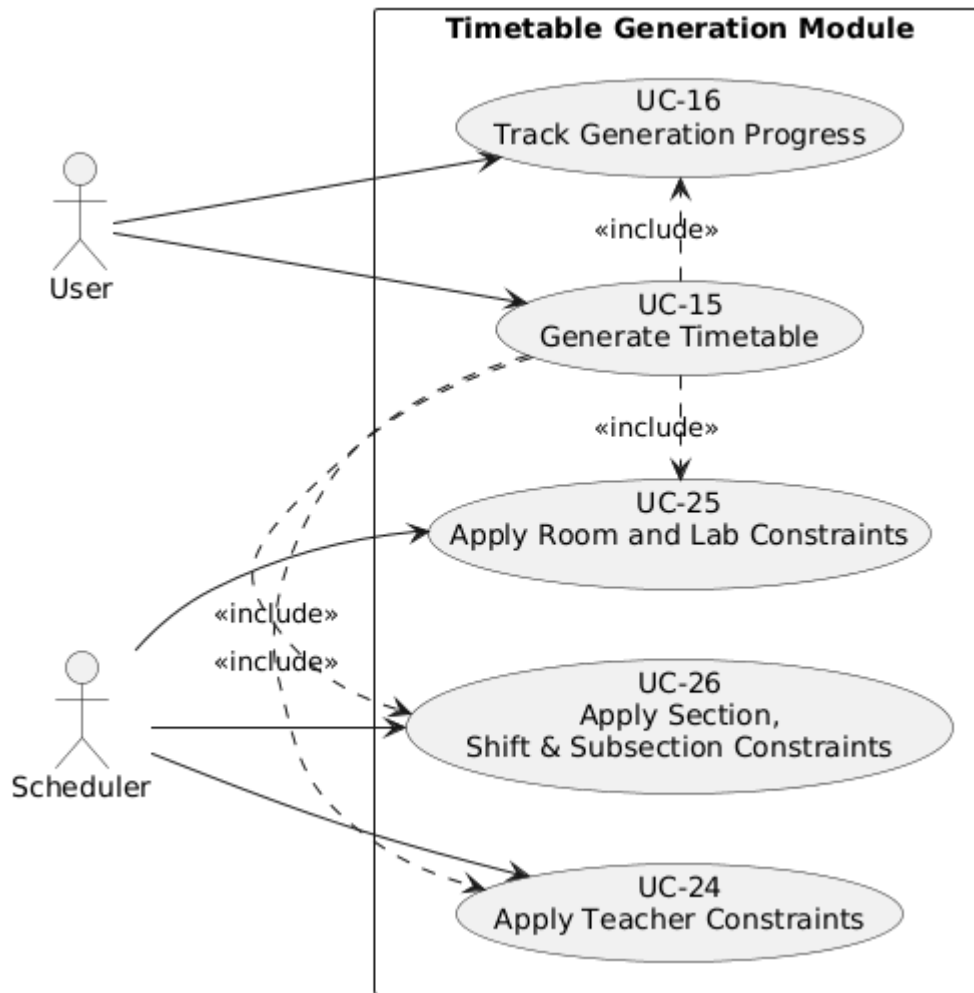
Use case 13:



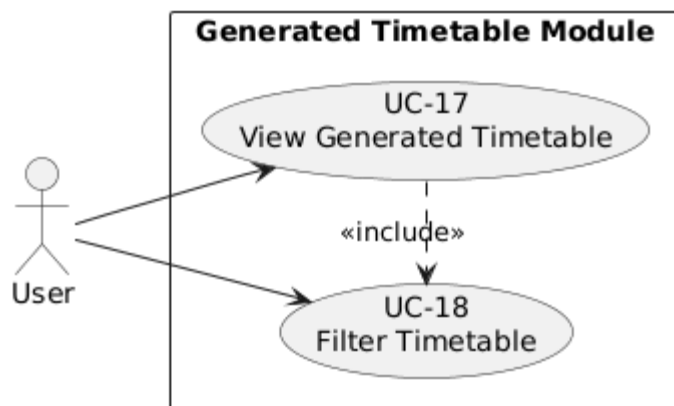
Use case 14:



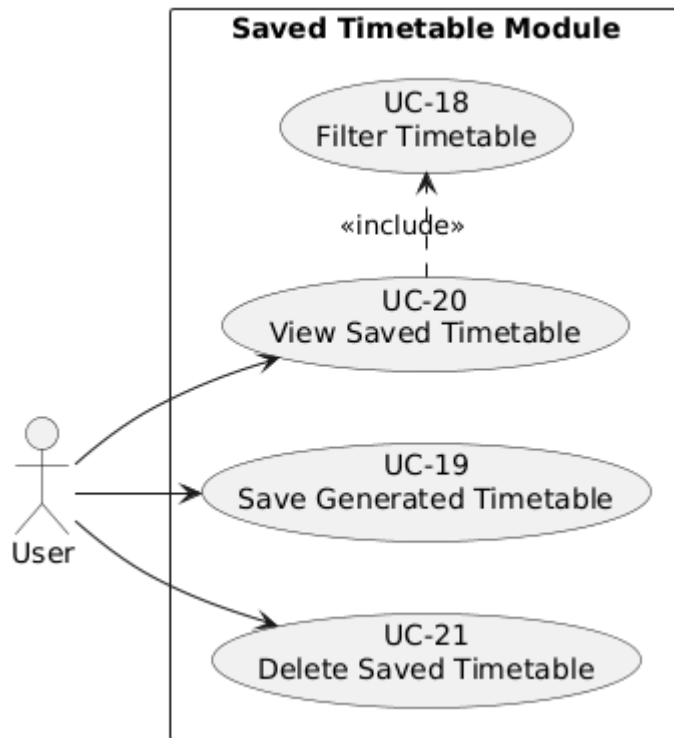
Use case 15, 16, 24, 25, 26:



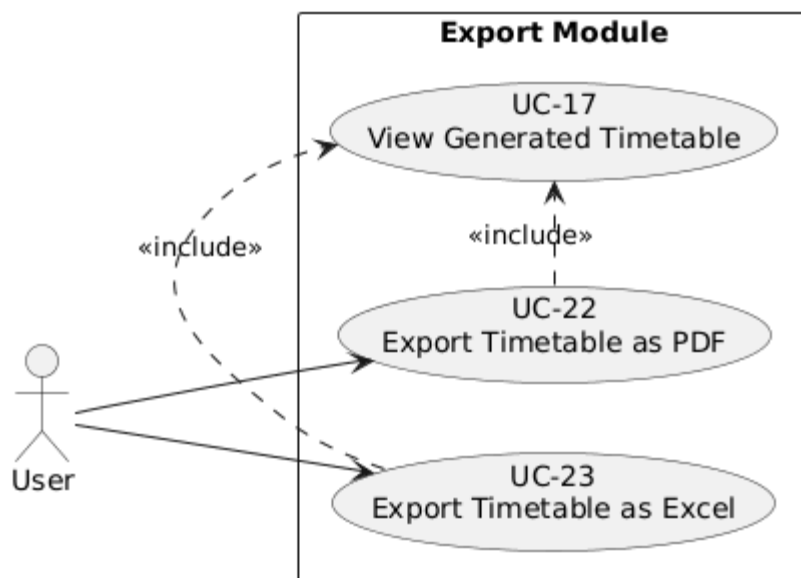
Use case 17, 18:



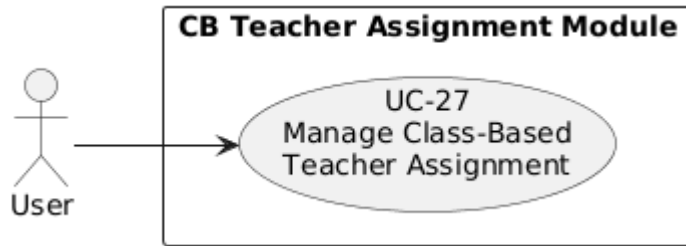
Use case 19, 20, 21:



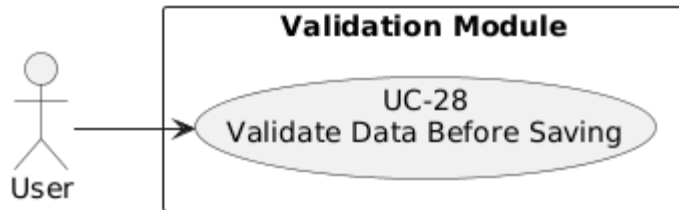
Use case 22,23:



Use case 27:



Use case 28:



2.7 Nonfunctional Requirements

2.7.1 Performance Requirements

- **Timetable Generation Time:**

The scheduler must schedule lectures of 150 class groups (scheduling over 4,500+ lectures per week) within **30 minutes or preferably less time** since the user initiates the generate timetables process.

- **Rationale:**

To gain advantages of the technology being built we aim to hard bound a time limit for generating timetables in order to provide difference in time utilization from the manual process currently being used.

- **API Response Time (Schedule Retrieval):**

API endpoints used for retrieving and displaying schedules (Teacher Schedule Viewer, Room Occupancy Grid, Section View) must respond within **2 seconds**.

- **Rationale:**

We require a responsive and progressive response from the system in order to deem it efficient.

- **Scalability:**

The system has to have the ability to tolerate more than double the data being used while testing and it should also be robust enough to handle all the data along with providing room to implement different constraints.

- **Rationale:**

This requirement ensures long-term usability and supports institutional growth as well as growth beyond the initial scope (other universities).

2.7.2 Safety Requirements

- **Data Integrity and Consistency:**

The system uses PostgreSQL as the framework of the main database and it needs to be

established that manual querying and overriding must not be maliciously used so that the integrity of data remains intact.

- **Safeguards:**

Various checks and validations need to be implemented at the backend and frontend wherever appropriate.

- **Conflict Prevention:**

The core scheduling engine, implemented as a **Python-based constraint satisfaction solver**, must strictly enforce all defined **Hard Constraints** (e.g., a teacher or room cannot be assigned to more than one lecture at the same time). Any violation of a hard constraint is considered a critical system failure.

- **Backup and Recovery:**

Recurring backups must be maintained of the PostgreSQL database and fall back must exist in case of a catastrophe, roll back to the system 4 hours.

2.7.3 Security Requirements

Security is mandatory to protect critical academic and administrative data.

User Authentication:

A secure authentication process through login is mandatory for all users of the system. The password will have to be kept in its encrypted format along with the use of the salt.

Authorization (Role-Based Access Control – RBAC):

The access controls will be based on roles, and should include:

- **Academic Coordinators:** Complete read/write access, including data entry using Excel and the form fields, production of the timetable, and the analysis of timetables generated.
- **Teachers/Administration:** Read-only access, limited to viewing the schedules.

Secure Communication:

All API calls and replies must use encryption using TLS.

Input Validation:

Inputs submitted by the user in the frontend (TypeScript) and in the server (backend in Python) must be validated against injection attacks, bad data format, XSS, and logic errors

2.7.4 Additional Software Quality Attributes

Adaptability: The system must be adaptive to any information entered into it.

Availability: The system must be easily available whenever it is needed.

Correctness: Timetables created must always be logical and should follow all the constraints.

Flexibility: The system must have flexibility for modifications as per the realities of life.

Maintainability: Maintenance of the system must be carried out at regular intervals to create an efficient system.

Portability: The system must be portable by running in common cloud computing services.

Reliability: The system must always be able to create an excellent schedule timetable with accuracy and reliability.

Reusability: The logic of scheduling must be reusable in other academic programs as well.

Robustness: The system must be fault tolerant and should handle incorrect or incomplete information as per the level of importance of the task.

Usability: The interface of the system must be user friendly and should be easily comprehensible for staff members without causing confusion.

Scalability: The system must be flexible enough to accommodate increase in the amount of data.

2.8 Other Requirements

- **Database Requirements:**

The database must be comprehensible and secure to handle large amounts of data querying while timetable generation process.

- **Reuse Objectives:**

The constraint satisfaction logic and overall system architecture are designed with the goal of streamlining scheduling across other BISE institutes as well with minimal adjustments.

Chapter 3. System Design

3.1 Application and Data Architecture

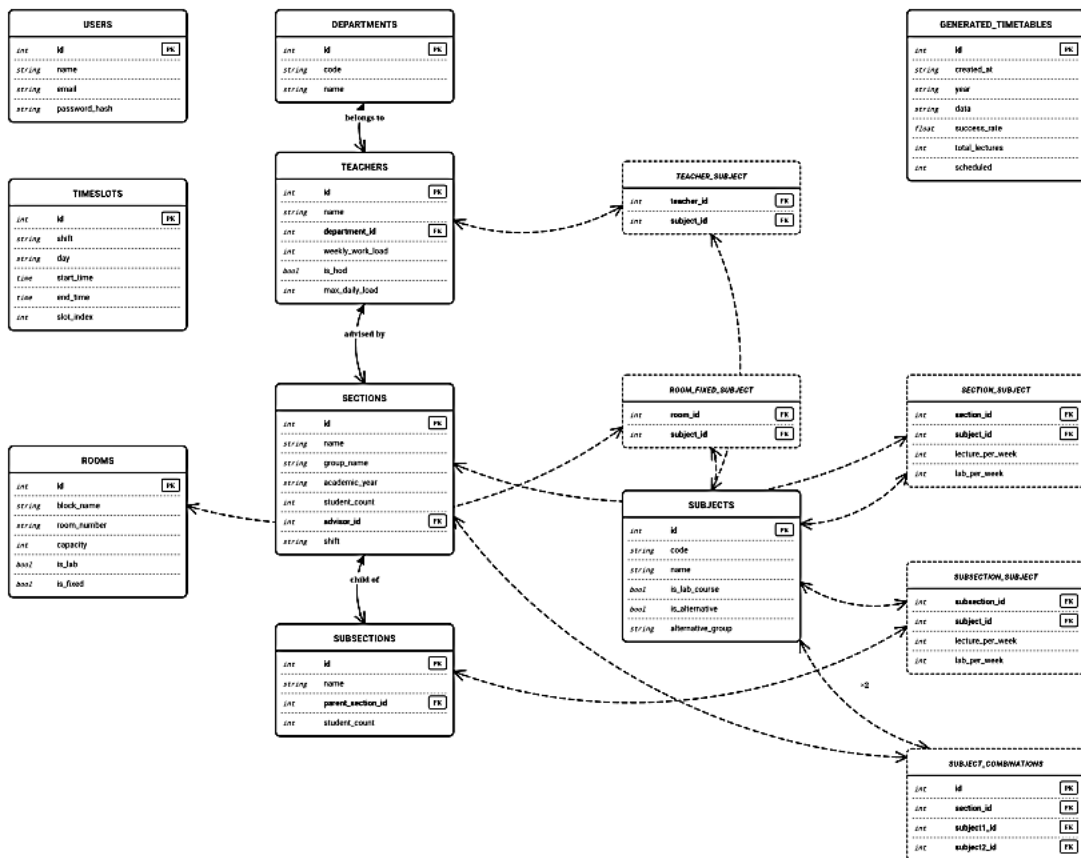
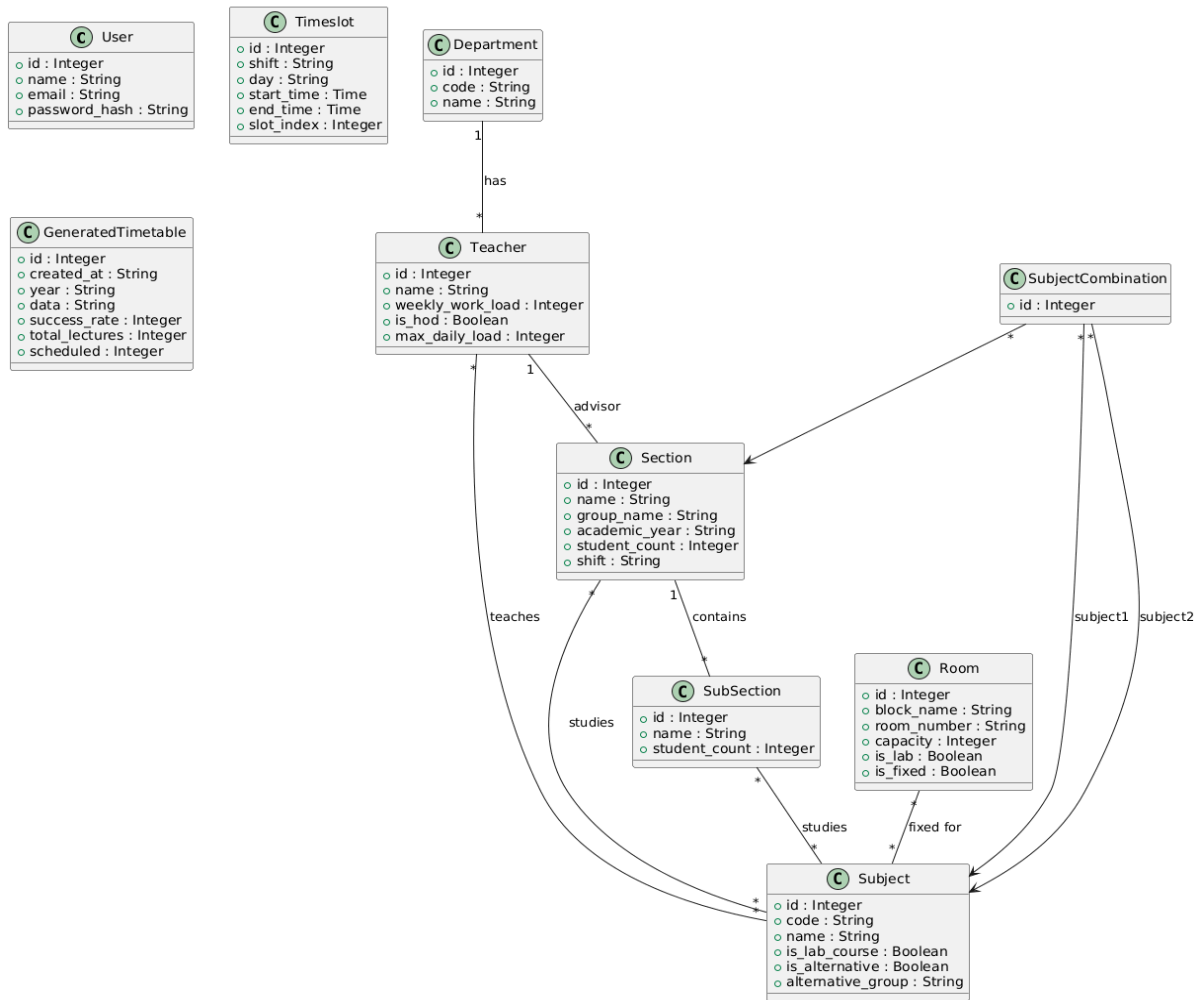
Our system has a **layered architecture** that contains presentation, logic of business, and data storage.

The model of the data is based up on the basic entity **TimetableEntry**, which represents the assignment of a subject to a particular time slot, room, and teacher.

For storing data in a persistent form, PostgreSQL database has been chosen, providing ACID compliant transactions that assure the integrity of data.

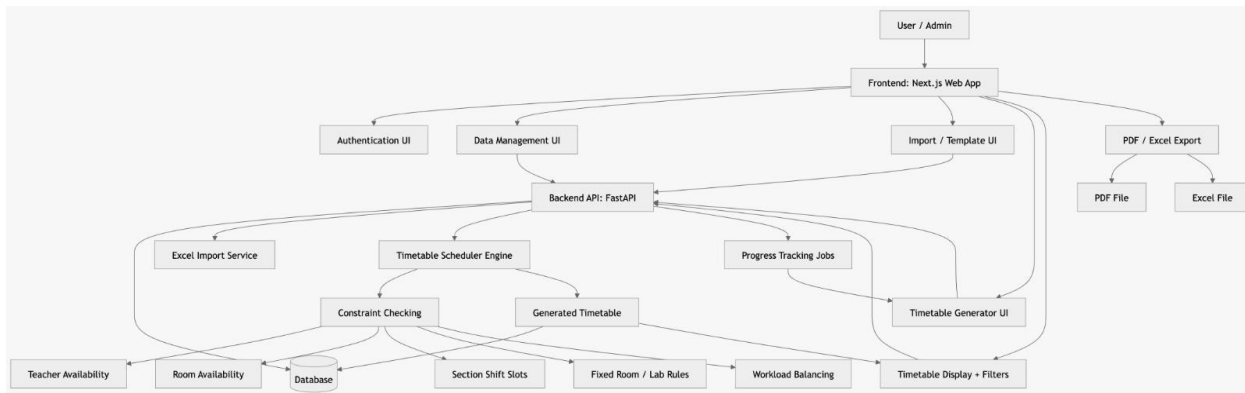
Core Entities Managed by the System

- **TimetableEntry: Scheduling of lectures or laboratories**
- **Teacher: Teacher details along with availability and weekly workload limit.**
- **Room: Room capacity, type (Lecture/Lab) and its availability**
- **Subject: It defines various attributes like number of lectures per week, laboratory requirement and ability to merge with other subjects.**
- **Groups/ Sections: Students' groups containing attributes like program stream (Pre-Medical, Pre-Engineering, ICS, Arts, ICom) year (1st, 2nd) and shifts (Morning, Afternoon).**



3.2 Interactions and Collaborations of Components

1. Academic Coordinator initiates creation of a timetable through Frontend (React.js/Next.js).
2. Frontend requests the creation of the timetable from the Python-based Backend by means of FAST API, all communication happens under protection provided by HTTPS/TLS protocol.
3. Backend gets current domain entities (Teachers, Rooms, Sections, Subjects and constraints that already exist) from PostgreSQL database using database access layers and transactions.
4. Obtained data is transferred into the custom Python-based Constraint Satisfaction engine which represents scheduling problem according to predefined hard and soft constraints.
5. Constraint solver executes scheduling algorithm that creates conflict free timetable, strictly fulfilling all Hard Constraints (for example, preventing teachers and rooms double booking), trying to optimize Soft Constraints (balanced load on teachers and efficient usage of rooms).
6. Created timetable is sent to the Backend which saves new TimetableEntry entities into the PostgreSQL database.
7. Obtained schedule is sent back to the Frontend.



3.3 System Architecture

Slotify uses a modular three-tier architecture to achieve scalability and maintenance. The technical architecture was designed to deal with the high computational demands of constraint-based scheduling.

Technical architecture comprises the following main components:

- **Frontend:** React.js and Next.js for fast UI responsiveness.
- **Backend:** A Python-based backend (FastAPI or Flask) providing a scalable and secure FAST API.
- **Scheduling Engine:** A Python-based Custom Constraint Satisfaction engine responsible for enforcement of hard constraints and optimization of soft constraints during timetable generation.
- **Database:** PostgreSQL for reliable relational database management.

3.4 Architecture Evaluation

A number of factors have been considered while choosing the required technology stack for our task in order to make sure that the system is capable of working in the real world and that it is able to cope with the scheduling problems of the Intermediate Program at Forman Christian College on a large scale basis. Due to the fact that there are many different classes, teachers, rooms, etc., the system needs to be equipped with a sophisticated scheduling algorithm that will provide accuracy, clarity, and flexibility of the process.

The Constraint Satisfaction Problem (CSP) serves as the foundation for the scheduling module of Slotify. This approach was selected instead of the general scheduling methods, which are based only on the metaheuristic algorithms (for example, Genetic Algorithms) due to the following reasons. The main advantage of the constraint satisfaction problem is the modeling of the hard constraints (non-duplicate usage of rooms/teachers) and optimization of the soft ones (balance of the workload and efficient resources' usage).

Backend was built on top of Python FAST API due to its flexibility, ease of implementation, and extensive support in data processing and optimization. Using Python will make it easy to integrate API layer and constraint solver, thus reducing overall complexity of the system.

PostgreSQL serves as our relational database because of its reliability, ACID support, and good performance when dealing with complex queries on structured datasets, such as timetable records, faculty loads, room allocation, etc. This ensures a reliable foundation for our timetable generator as well as real-time retrieval of schedules.

In order to provide fast and user-friendly interface, we use React.js and Next.js frameworks to build frontend of our application. Component based approach together with PWA functionality makes it easy to visualize schedules on both desktop and mobile devices.

3.5 Interface of Component with External Entities

Since Slotify is an in-house scheduling system, it does not need any interaction with external third-party systems like NADRA, banking networks, or credit card verification systems. Only interaction from an external point of view will be between user's client (Frontend) and application server (Backend).

- **User/Client ↔ FAST API:** React.js/Next.js Frontend communicates with CSP Python Backend through FAST API for performing all database operations including timetable creation and schedule retrieval (teacher-wise, class-wise, room-wise).
- **Security:** All data communications are secure and encrypted through HTTPS/TLS to keep all the administrative data safe.
- **Data Integrity:** FAST API works as an entry point, ensuring all the data that gets stored in PostgreSQL follows the ACID property.

3.6 Screenshots/Prototype Screens:



Slotify
Welcome back

EMAIL

PASSWORD

Sign In

Don't have an account? Sign up



Slotify
Create your account

FULL NAME

EMAIL

PASSWORD

Create Account

Already have an account? Sign in

Saved Timetables

Generated timetables saved with a custom name appear here.



No saved timetables yet.
Generate one and save it to see it here.

DEPARTMENTS17

SUBJECTS33

TEACHERS211

ROOMS88

TIMESLOTS107

SECTIONS84

SUBSECTIONS2

Data Completion

100%

✓ All data complete – ready to generate

+ Add Department

+ Add Subject

+ Add Teacher

+ Add Room

+ Add Timeslot

+ Add Section

Excel Import / Templates

Use the template headers exactly as manual inputs.

Departments

Template

Import

Subjects

Template

Import

Teachers

Template

Import

Rooms

Template

Import

Timeslots

Template

Import

Sections

Template

Import

Subsections

Template

Import

+ Add Department

+ Add Subject

+ Add Teacher

+ Add Room

+ Add Timeslot

+ Add Section

Excel Import / Templates

Use the template headers exactly as manual inputs.

Departments

Template

Import

Subjects

Template

Import

Teachers

Template

Import

Rooms

Template

Import

Timeslots

Template

Import

Sections

Template

Import

Subsections

Template

Import

Q Search departments, subjects, teachers, rooms, timeslots, sections, subsections...

Departments			17	All
CODE	NAME			
CHEM	Chemistry			
CRST	Christian Studies			

Subjects				33	All
CODE	NAME	TYPE			
Eng	English	Theory			
Urdu	Urdu	Theory			

Teachers				211	All
NAME	DEPT	LOAD			
BT1	Biology	20			
BT2	HOD Biology	12			

Add Department

CODE

e.g. CS

NAME

Full name

Cancel

Save

Add Subject

CODE

e.g. MATH101

NAME

Subject name

☐ Lab Course

Cancel

Save

Add Teacher

NAME

Full name

DEPARTMENT

Select Department

WEEKLY LOAD

20

MAX DAILY

6

☐ Is HOD

SUBJECTS TAUGHT

Eng – English
Urdu – Urdu
Isl – Islamiyat
TQ – Translation of Quran
Eth – Ethline

Hold Ctrl to select multiple

Cancel

Save

Add Room

BLOCK

Block name

ROOM NO.

Room number

CAPACITY

45

☐ Is Lab

☐ Fixed Room

Cancel

Save

Add Timeslot

Bulk (Multiple Days)

Single Day

SHIFT

MORNING

DAYS

☐ MONDAY

☐ WEDNESDAY

☐ FRIDAY

☐ TUESDAY

☐ THURSDAY

☐ SATURDAY

START TIME

09:00 am

END TIME

10:00 am

SLOT INDEX

0

Cancel

✓ Save

Add Section

SECTION NAME

e.g. IE1

GROUP

Select Group

ACADEMIC YEAR

Select Year

STUDENT COUNT

45

SHIFT

MORNING

ADVISOR (OPTIONAL)

No advisor

SUBJECTS & CREDITS

No subjects added yet

Add Subsection

Adding subsection to Section ID: 10

SUBSECTION NAME

e.g. IE1-A

STUDENT COUNT

20

SUBJECTS & CREDITS

No subjects added yet

+ Add Subject

Cancel

✓ Save

Timetable Management
Dashboard
Data
Gene

ite Timetable

SHIFT

YEAR

GROUP

All Shifts

All Years

All Groups

Generate Timetable

3.7 Other Design Details

Constraint Modeling and Optimization

The fundamental approach of Slotify is based on constraint modeling and optimization which is done through customized Python Constraint Satisfaction Problem (CSP) engine. The problem of academic scheduling can be defined as the optimization problem where there is a need for creating possible schedules while complying with the set of rigid institutional policies and at the same time optimizing the quality of created timetables.

Instead of using the black box solver, the approach is more transparent as it incorporates scheduling variables, their domains and constraints, thus allowing easy enforcement of the rules and adjustment to the policies of Forman Christian College.

The enforcement of institutional requirements is done through dividing all constraints into two groups:

Hard Constraints

Hard constraints stand for those rules which can't be violated no matter what. They include:

- Avoiding double booking teachers and rooms.
- Setting the maximum work limit for teachers (18-20 hours per week).
- Room capacity and type compatibility.
- Scheduling labs to run as double period classes during two time slots.

If there is a violation of any hard constraint, then the solution is to be rejected. Any violation of a hard constraint is considered a **critical system failure** and results in the rejection of the generated solution.

Soft Constraints

Preferences related to soft constraints help make sure that the generated timetable is as user-friendly and good quality as possible. In order to accomplish this, the optimization will try to:

- **Balance workload between teachers.**
- **Minimize idle time of teachers.**
- **Optimize time/resource use.**

Where possible, soft constraints can be loosened, but the solver always tries to satisfy them, while never breaking any hard constraints.

Component and External Interfaces

Slotify adopts a service-oriented architecture that is modular in nature, whereby services are connected using clear interface definitions, primarily based on a FAST API created by the Python backend.

Frontend Interface

The React.js/Next.js frontend only connects to the FAST API on the backend via HTTPS and provides the following functionality:

- **Schedule view (teacher-wise, class-wise, and room-wise).**
- **Creation and manipulation of core domain objects like Teachers, Rooms, Subjects, and Sections.**
- **Generation of timetables**

Database Interface

The Python backend accesses the PostgreSQL database using database access layers and transaction management techniques. All operations performed by the system adhere to ACID properties, to ensure data integrity while generating timetables or when performing manual updates of the timetable. The database structure has been optimized to fetch many TimetableEntry objects.

Scheduling Engine Interface

The constraint satisfaction module is embedded within the Python backend. It takes inputs in the form of domain objects that have been validated and performs constraint modelling and optimization and produces an error-free timetable. The timetable generated is then stored back to the database.

Chapter 4. Test Specification and Results

4.1 Test Case Specification

TC-1 – TBD Teacher Auto-Generation

Field	Value
Identifier	TC-1
Related requirements(s)	Teacher Allocation Module; Main.py (Lines 1–50)
Short description	Automatically generates TBD teachers when no qualified teacher exists for a subject
Pre-condition(s)	Subject exists but no teacher is assigned to it
Input data	<code>neededSubjects = {AI-101}; subjectTeachers.get(AI-101) = ∅</code>
Detailed steps	Scan subjects → check teacher mapping → detect missing → create TBD teachers → add subject → append to teacher list
Expected result(s)	Warning printed and TBD teachers created
Post-condition(s)	Subject becomes schedulable with violation flag
Actual result(s)	PASS – Works correctly but introduces hard constraint violations

Field	Value
Identifier	TC-2
Related requirements(s)	Python constraint file
Short description	Subsections must have lessons in the same timeslot

Pre-condition(s)	Section has multiple subsections
Input data	Lesson A = 8AM, Lesson B = 9AM
Detailed steps	Run solver → compare timeslots
Expected result(s)	Hard penalty if mismatch
Post-condition(s)	Solver aligns timeslots
Actual result(s)	PARTIAL PASS – Constraint works but conflicts remain

Field	Value
Identifier	TC-3
Related requirements(s)	Room fixed constraint
Short description	Fixed rooms allow only specific subjects
Pre-condition(s)	Room is fixed
Input data	Physics lesson in Chemistry lab
Detailed steps	Assign lesson → evaluate
Expected result(s)	Hard violation
Post-condition(s)	Lesson reassigned
Actual result(s)	PASS

Field	Value
-------	-------

Identifier	TC-4
Related requirements(s)	Section.addGroupSubject()
Short description	Advisor auto-assigned to CB subject
Pre-condition(s)	Advisor exists
Input data	Subject code = CB
Detailed steps	Add subject → check advisor subjects
Expected result(s)	Advisor assigned automatically
Post-condition(s)	Eligible for teaching
Actual result(s)	PASS

Field	Value
Identifier	TC-5
Related requirements(s)	python-constraint
Short description	Solver stops after time/unimproved limit
Pre-condition(s)	Large dataset loaded
Input data	60 min total, 15 min unimproved
Detailed steps	Run solver → monitor time
Expected result(s)	Stops within limits
Post-condition(s)	Best solution returned

Actual result(s)	FAIL – Slow, incomplete solutions
------------------	-----------------------------------

Field	Value
Identifier	TC-6
Related requirements(s)	Workload constraint
Short description	Teacher load must not exceed limit
Pre-condition(s)	Workload = 20
Input data	22 lessons assigned
Detailed steps	Assign → evaluate
Expected result(s)	Hard violation
Post-condition(s)	Redistribution
Actual result(s)	PARTIAL PASS

Field	Value
Identifier	TC-7
Related requirements(s)	Room.isLab constraint
Short description	Labs must be in lab rooms
Pre-condition(s)	Lab subject
Input data	Lab in lecture hall

Detailed steps	Assign → check
Expected result(s)	Hard violation
Post-condition(s)	Move to lab
Actual result(s)	PASS

Field	Value
Identifier	TC-8
Related requirements(s)	Section shift constraint
Short description	Lessons must fit shift
Pre-condition(s)	Limited timeslots
Input data	Lesson outside shift
Detailed steps	Assign → check
Expected result(s)	Hard violation
Post-condition(s)	Move to valid slot
Actual result(s)	PASS

Field	Value
Identifier	TC-9
Related requirements(s)	TBD generation

Short description	Replace TBD teachers
Pre-condition(s)	TBD teachers exist
Input data	TBD assigned
Detailed steps	Run solver
Expected result(s)	Replace or flag
Post-condition(s)	No TBD or warnings
Actual result(s)	FAIL – Unresolvable

Field	Value
Identifier	TC-10
Related requirements(s)	python-constraint
Short description	Solver gets stuck
Pre-condition(s)	Complex dataset
Input data	Standard dataset
Detailed steps	Run solver
Expected result(s)	Escape local optima
Post-condition(s)	Improved score
Actual result(s)	FAIL

Field	Value
-------	-------

Identifier	TC-11
Related requirements(s)	All constraints
Short description	Final solution has violations
Pre-condition(s)	Full dataset
Input data	All constraints enabled
Detailed steps	Validate final solution
Expected result(s)	Zero violations
Post-condition(s)	Fully feasible
Actual result(s)	FAIL

Summary of All Test Results

Module Name	Test Cases Run	Defects Found	Corrected	Remaining
Teacher Allocation	TC-1, TC-9	2	0	2
SubSection Constraint	TC-2	1	0	1
Room Constraints	TC-3, TC-7	0	0	0
Advisor System	TC-4	0	0	0
Solver Performance	TC-5, TC-10	2	0	2
Workload Constraint	TC-6	1	0	1
Shift Constraint	TC-8	0	0	0
Overall System	TC-11	1	0	1

Complete System	TC-1 → TC-11	7	0	7
------------------------	--------------	----------	----------	----------

4.2 Defect Summary & Recommendations

Defect ID	Description	Severity	Root Cause	Recommended Fix
D-001	Slow solver / no feasible moves	High	Weak search strategy	Add Tabu Search / Late Acceptance
D-002	Unsatisfiable constraints	High	Over-constrained system	Relax soft constraints
D-003	Persistent TBD violations	Medium	Missing teachers	Pre-validation before solving
D-004	Subsection timing failures	Medium	Resource conflicts	Increase rooms/teachers
D-005	Workload imbalance	Low	Poor weight tuning	Adjust constraint weights

Chapter 5. Conclusion and Future Work

5.1 Project Summary

Slotify effectively transforms Forman Christian College into using an automated timetable generating mechanism from the inefficient manual method. This involves creating a Python backend using the help of a constraint satisfaction problem (CSP) solver, which ensures all hard constraints of the organization are adhered to while optimizing soft constraints for better quality timetables.

5.2 Problems Faced

The project got several challenges during the development lifecycle:

Technical Challenges:

Managing the huge scheduling task that had more than 4,500 weekly class schedules involved handling the complex set of constraints such as teachers' workload, classroom capacity, shift constraints, and laboratory classes.. It was a difficult task to develop a good constraint solver program in Python that would produce a timetable within a reasonable period of time.

Resource and Operational Challenges:

Issues pertaining to scalability, deployment, and performance came up during the process of development. Determining the right cloud infrastructure and performance of the system with regard to large data sets was a major issue

Challenges:

One of the most difficult aspects to address was to incorporate decision-making, based on practical experiences of academic coordinators, in the form of algorithmic restrictions. Another issue to consider was the constant changes that would come about in academic data.

Key Learning Outcomes

- Hands-on experience in creating a **Constraint Satisfaction Problem (CSP)** schedule management system using **Python**
- Better comprehension of various optimization methods, constraint modeling, and systems design
- Better proficiency in developing a complete stack system with a **backend** developed in **Python**, a database in **PostgreSQL**, and frontend in React and **Next.js**
- Better skills in formulating practical real-world institutional rules in terms of computational language
- Better team working skills along with better system integration.

5.3 Future Work

Future iterations of the system may include:

- Development of a specific mobile app or improvement of the PWA to increase accessibility
- Integration with current Student Information Systems (SIS) to automate data synchronization
- Adoption of university-level scheduling that will include scheduling based on electives and multi-campus schedules
- Application of even more complex optimization methods to enhance the quality of solutions
- Creation of analytics dashboards to assist in workload forecasting and resource planning

References

1. S. Thakare, T. Nikam, and M. Patil, "Automated Timetable Generation Using Genetic Algorithm," International Journal of Engineering Research & Technology, vol. 9, no. 7, 2020.
2. "Automated Timetable Generator Using Genetic Algorithm," International Journal of Research in Advent Technology, vol. 1, no. 10, 2013.
3. OptaPlanner, "School Timetabling Use Case," Red Hat Build of OptaPlanner 8.29 Documentation.
4. Red Hat Build of OptaPlanner 8.29 Documentation, "OptaPlanner on Spring Boot: a school timetable quick start guide,"
5. Thakare, S., et al. (2020). "Automated Timetable Generation Using Genetic Algorithm."

Appendix A Glossary

Term / Acronym	Category	Definition
ACID Properties	Technical	A set of properties (Atomicity, Consistency, Isolation, Durability) that ensure reliable database transactions.
API	Technical	Application Programming Interface; the bridge enabling communication between the React frontend and CSP Python backend.
Constraint (Hard)	Optimization	A mandatory rule that must not be violated (e.g., a teacher cannot be in two rooms at once).
Constraint (Soft)	Optimization	A preferential rule that improves quality (e.g., minimizing gaps in a teacher's day) but may be violated.
PWA	UI/UX	Progressive Web App; a web application that provides app-like functionality and mobile compatibility.

Appendix B Deployment/Installation Guide

The Slotify solution comprises a three-layer architecture involving a backend built using Python constraint solving, a frontend using React/Next.js and SQLite database. The application is containerized via Docker to allow for portability across different cloud environments such as AWS, GCP, and Railway

- **Source code:** Entire Slotify codebase (Backend and Frontend).
- **Node.js & npm/yarn:** For frontend build using React/Next.js framework.
- **Python 3.10+ :** For backend and constraint solver.

Steps for Backend & Engine Deployment (CSP Python)

1. **Configuration:** Set up the environment variables with PostgreSQL database connection settings (URL, username, and password).
2. **Environment Setup:** Install all required packages in Python.
3. **Rest API:** Run the Python REST API (FAST API server).
4. **Containerization (Optional):** Build the Python backend container using Dockerfile and ensure that it can connect with the PostgreSQL as well as frontend.

Steps for Frontend Deployment

1. **Configuration:** Configure frontend variables (e.g., `.env.production`) with the deployed URL of Python backend API.
2. **Build:** Create the production build for Next.js.
3. **Serve:** Serve static assets using web server/hosting.

Step 3: Post-Installation/Initial Data Load.

Access System: Go to the deployed URL and log in using given Academic Coordinator account.

Data Input: Use administrative interfaces (UC-5 to UC-11) to add institutional data.

Verification: Ensure system readiness by checking that the CSP successfully generate a complete timetable (for any reference UC-4).

Appendix C User Manual

This manual provides instructions for the three primary user classes of the Slotify system.

1. Academic Coordinators (Full Access)

1. Step 1: Data Configuration

Go to the management panels (Teachers, Rooms, Subjects, Groups, Shifts) to input/ update/ delete institutional data.

2. Step 2: Timetable Generation

Go to the “Generate” section and click **Generate Timetable** to activate the Python-based constraint satisfaction engine.

3. Step 3: Review and Override

Use the grid-based interface to review generated schedules. Manual adjustments can be made where necessary using override controls.

4. Step 4: Export

Export the finalized timetable in PDF or Excel format for institutional use.

Appendix D Student Information Sheet

Roll No	Name	Email Address (FC College)	Personal Cell Phone Number
261939880	Abial Zafar	261939880@formanite.fccollege.edu.pk	0305-4506340
261937391	Alisha Saleem	261937391@formanite.fccollege.edu.pk	0324-4181851
261939549	Sheril Bobby	261939549@formanite.fccollege.edu.pk	0313-4302318
261940660	Muhammad Zeeshan	261940660@formanite.fccollege.edu.pk	0321-6223535

Appendix E Plagiarism Free Certificate

This is to certify that, I am _____ S/D/o _____, group leader of FYP under registration no _____ at Computer Science Department, Forman Christian College (A Chartered University), Lahore. I declare that my Final year project report is checked by my supervisor and the similarity index is _____% that is less than 20%, an acceptable limit by HEC. Appendix F is attached along with the report. As far as I know and believe, there is nothing in the report that has been taken from somewhere else without due credit being given in the report.

Date: _____ Name of Group Leader: _____ Signature: _____

Name of Supervisor: _____ Co-Supervisor (if any): _____

Designation: _____ Designation: _____

Signature: _____ Signature: _____

Senior Project Management Committee Representative: _____

Signature: _____

Appendix F Plagiarism Report

FYP

ORIGINALITY REPORT

15%

SIMILARITY INDEX

14%

INTERNET SOURCES

1%

PUBLICATIONS

12%

STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Higher Education Commission Pakistan Student Paper	6%
2	www.coursehero.com Internet Source	3%
3	Submitted to University of East London Student Paper	2%
4	abdullah-mehtab.github.io Internet Source	1%
5	Submitted to University of Houston, Downtown Student Paper	<1%
6	Submitted to Arab Open University Student Paper	<1%
7	docs.redhat.com Internet Source	<1%
8	www.ssgmce.ac.in Internet Source	<1%
9	www.fccollege.edu.pk Internet Source	<1%
10	Submitted to University of Greenwich Student Paper	<1%
11	www.allscientificjournal.com Internet Source	<1%
12	Submitted to Islington College, Nepal Student Paper	<1%

13	Submitted to South Bank University Student Paper	<1 %
14	Submitted to Technische Hochschule Deggendorf Student Paper	<1 %
15	Prerna Dubey, Khushi Gupta, Prachi Dhanawat, Mansi Prashant Awari. "Automatic Timetable Generation", Institute of Electrical and Electronics Engineers (IEEE), 2025 Publication	<1 %
16	Submitted to Rivier University Student Paper	<1 %
17	irigs.iiu.edu.pk:64447 Internet Source	<1 %
18	www.itc.nl Internet Source	<1 %
19	www.journalwjarr.com Internet Source	<1 %
20	uniquepaperhelp.com Internet Source	<1 %

Exclude quotes Off
Exclude bibliography Off

Exclude matches Off